A 215-State ZFC Consistency Machine with a New Enumeration and One-Head Stack Credit

Research packet for Vladimir Reshetnikov's ProveIt project

20 October 2026 (Pacific time)

Abstract

We construct an explicit two-symbol Turing machine with 215 working states, including blank-tape initialization, and a separate halt. Its nonhalting is equivalent to Con(ZFC) by ordinary mathematical reasoning formalizable in ZFC, using the pinned ProveIt numerical-calculus calibration as an explicit inherited theorem. The construction assigns command tag zero to modus ponens and tag one to the previously proposed composite transfer command. This changes the exhaustive enumeration and the actual transition graph. A new diagonal argument proves an exact one-head bound for conservative stack accounting from candidate cutoff one, compared with two for the earlier ordering. The complete source has 4,775 primitive occurrences and 201 distinct subtrees. A fresh whole-tape symbolic induction check proves all 6,209 guarded cases; an independently replayed 68,442-transition blank prefix is matched to an internal point of a sound synthetic AX computation. The graph is distinct from 47 prior 215-state tables under state renaming and global reflection. On the logical side, a finite Hilbert derivation shows that only one orientation of quantifier commutation is needed. Shorter transfer allocations and other attempted reductions are documented with their exact failure boundaries. No complete 214-state construction or new proof-assistant endpoint is claimed.

Contents

1 Result, conventions, and contribution 3
1.1 What is new and what is inherited . . . 3

2 Inherited calculus and the treatment of junk codes 4

3 Three commands and their exact wire format 4
3.1 Composite transfer and its two guards . . . 5
3.2 Modus ponens and axioms . . . 5
3.3 Soundness with a protected stack prefix . . . 6

4 A complete derivation compiler 6
4.1 Truth and parameter setting . . . 6
4.2 Axiom selection . . . 7
4.3 Generalization . . . 7
4.4 Modus ponens and its zero alias . . . 7
4.5 Cofinal enumeration and completeness . . . 7

5 One protected head suffices for the new schedule 8

6 Exact source and whole-tape implementation 9
6.1 Frames . . . 10

7 Blank startup and the arbitrary tail 10

8 A calculus reduction: one orientation of quantifier commutation 11
8.1 Definitions and hypotheses . . . 11
8.2 Finite derivation of the opposite direction . . . 12
8.3 Certificate and separate audit . . . 13
8.4 Scope for machine optimization . . . 13

9 Research toward 214 and exact failure boundaries 14
9.1 Runtime folding and initialization . . . 14
9.2 A shorter transfer and context-sensitive primitive entries . . . 14
9.3 Shorter delimiter scanner . . . 15
9.4 Finite local-fold searches . . . 15

10 Reproduction and proof status 15

A Complete dense transition table 16

1 Result, conventions, and contribution

Theorem 1.1 (Exact delivered machine)**.** *Let* T215 *be the dense transition table in* `machine/t215.tm`*. Its working states are* $0, \ldots, 214$*, its halt label is 215, its initial state is 0, and its initially blank, two-way infinite tape has alphabet* $\{0, 1\}$*. Every transition moves exactly one cell left or right. Relative to the inherited numerical-calculus theorem stated in Section 2, the proofs and exact certificates in this packet establish, by ordinary reasoning formalizable in ZFC,*

$$\mathrm{ZFC} \vdash \big(\mathrm{Halt}(\mathsf{T215}, \mathrm{blank}) \leftrightarrow \neg\, \mathrm{Con}(\mathrm{ZFC})\big),$$

and therefore

$$\mathrm{ZFC} \vdash \big(\neg\, \mathrm{Halt}(\mathsf{T215}, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{ZFC})\big).$$

The theorem is a statement about one literal finite transition table. It does not assert the machine's unconditional nonhalting. The halt state is excluded from the working-state count; initialization states are included. There is no prewritten input, larger alphabet, external initialization, or finite search cutoff.

The requested 214-state target remains unmet. The delivered result uses the alternative allowed in the request: a materially different 215-state implementation. The change is to the literal command encoding and the resulting enumeration: modus ponens has tag zero and transfer has tag one. Candidate zero is therefore MP, candidate one is transfer, and candidate two is MP. This ordering improves the conservative stack-debt bound from two to one and changes eight transition rows of the earlier transfer table. The included exact rooted-graph comparisons establish that the new table is not a state renaming or a global reflection of any of the 47 distinct prior tables examined. The inherited transfer semantics and arithmetic macros are not claimed as new.

Quantity	Delivered primary machine
Working states / transition rows	215 / 430
Primitive source occurrences	4,775
Distinct source subtrees	201
Maximum source depth	44
Maximum consecutive zeros in a source address	6
Guarded arbitrary-natural runtime cases	6,209
Symbolic rule applications	1,610,781
Minimum constant in a duration polynomial	10
Blank transitions to the matched frame	68,442
Protected theorem heads required after the AX completion	1
Protected theorem heads supplied by the AX completion	10

1.1 What is new and what is inherited

The base repository is pinned at commit `ead95b5b336818a53c89b8c3d5249822e0570939` [1]. At that snapshot, the ordinary K215 proof is distinguished from still-unchecked exact-table Lean/Rocq proof scripts. This packet does not change that formal-verification status.

The transfer proof language and its strong derivation compiler were proposed in the earlier transfer research [4]. That research discussed the direct clear/copy/pop source, then delivered a composite DROP/NIP/NIP implementation. We credit those ideas explicitly. The contributions here are the new command encoding and its scheduler theorem, a complete exact table with fresh runtime evidence

and blank-start connection, the separately proved one-orientation quantifier-commutation interface, and concrete obstructions to several further reductions. No priority claim is made for the pre-existing transfer idea, Cantor codec, dominated-rho8 theorem, or binary architecture.

The source-DAG size and the machine-state count are different quantities. In particular, a 201-node source does not mean a 201-state Turing machine. Shared arithmetic routines, decoder roles, and initialization determine the complete physical cost.

2 Inherited calculus and the treatment of junk codes

Let

$$P(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \qquad I(a,b) = 4P(a,b), \qquad Q_i(a) = 4P(i,a) + 2.$$

Write $a \to b = I(a,b)$, truth $T = 0$, and falsity $F = 1$. The exact alias $I(0,0) = 0$ is retained. There is no global premise that a natural code decodes to an ordinary first-order formula.

We use the accepted dominated numerical calculus $\mathcal{K}_{\leq}$. Its inherited axiom interface consists of truth, seven native logical rows, the unchanged gated set-theory bundle, and

$$L(A,B,C,U) = ((C \to U) \to A) \to ((A \to C) \to (B \to C)), \qquad U \leq A.$$

Its inference rules are literal numerical modus ponens and theorem-only generalization. The permitted bundle is kept exactly, including its gate: it is disabled when $C = A$ or $C = B$. We do not weaken support-bearing conjuncts or assume arbitrary substitution into the bundle.

Inherited calibration theorem. The pinned ProveIt proof chain establishes equivalence of finite-refutation existence:

$$\mathcal{K}_{\leq} \vdash 1 \quad \Longleftrightarrow \quad \text{ZFC has a finite formal refutation.}$$

The dominated-rho8 result preserves every native theorem code: its 3,728-node schematic certificate derives unrestricted rho8 from the dominated instances, and the converse is immediate [3]. This is an explicitly imported theorem of the present packet, rather than a new attempt to reprove the whole set-theoretic interpretation. The predecessor proof, finite certificate, provenance, and needed immediate source dependencies are included or precisely pinned.

Our new compilation manipulates all natural numbers directly. Soundness does not follow from a syntactic type check; it follows from an invariant that the formula-bearing fields contain theorems of $\mathcal{K}_{\leq}$. Every raw candidate number is allowed. Malformed or unconventional codes may be created, but every output used as a theorem has a justified derivation. This directly uses the freedom in the user's problem formulation.

The following elementary coding facts are repeatedly used. Cantor pairing is a bijection on $\mathbb{N}^2$, including $P(0,0) = 0$, and dominates both arguments. Every implication code is divisible by four. Every quantifier code is positive and congruent to two modulo four. Thus implication codes and quantifier codes are never falsity 1, and a quantifier code cannot equal any implication code.

3 Three commands and their exact wire format

At a command boundary, let (t, w, d, h) denote current top, numerical stack, additive parameter, and cached formula. Write $[x; v] = P(x, v)$ and $[x_1, \ldots, x_k; v]$ for right-nested pairing. Stack notation

does not assume a free-list datatype: every statement below uses the identity $P^{-1}(P(x,v)) = (x,v)$, including zero aliases.

3.1 Composite transfer and its two guards

The delivered transfer command is

$$\mathsf{S}(t,[x;v],d,h) = \begin{cases} \text{halt}, & h = 1 \text{ or } t = 1, \\ (h,v,d,x), & h \neq 1 \text{ and } t \neq 1. \end{cases} \tag{1}$$

Its literal implementation is the inherited 78-primitive composite DROP; NIP; NIP. Both guards are essential to an accurate statement of this implementation. An earlier compiler presentation specified only the cache guard; the included composite-transfer review corrects that specification.

For the compiler calculations below, the single-cache-guard transfer has the same nonhalting map. The composite is an early-halt refinement: whenever the extra top guard fires it has already exposed a theorem candidate equal to falsity, while the compiler's strong contract explicitly permits an earlier halt. Every calculation that reaches its stated output is unchanged. For soundness, both the top and the cache are theorem-valued; for completeness, an earlier halt is an acceptable success.

We also tested a 35-primitive clear/copy/NIP implementation of the single-guard interface. Its initial 215-state allocations failed the full runtime proof in a concrete register-selection context. They are experimental negative controls, not alternative verified consistency machines.

3.2 Modus ponens and axioms

For $y = d + x$, define

$$\mathsf{MP}(x)(t,w,d,h) = \begin{cases} (y,[0;w],y,0), & I(t,y) = h, \\ (0,w,y,0), & I(t,y) \neq h. \end{cases} \tag{2}$$

The successful zero push is intentional. It is retained in the primary machine and used by the completeness compiler.

For $\mathsf{AX}(c;x,y)$, put $U = d$, $A = d + x$, $B = h + y$, and $C = c$. The command returns

$$\big(G(A,B,c),[R_1,\ldots,R_7,L(A,B,c,U),Q_U(h),t;w],0,0\big). \tag{3}$$

The seven R_j have the inherited native arguments. The formula G is the full enabled bundle or truth 0. In particular $G = 0$ if $c = A$ or $c = B$. The old cache is used in $Q_U(h)$ before the additive field y is read. The first nine emitted heads and every possible gate value differ from 1; the tenth head is the saved old top, which is unrestricted. There are exactly ten new stack heads, including that saved old top.

The command fields in the primary construction are

$$\text{fields}(\mathsf{MP}(x)) = [0,x], \qquad \text{fields}(\mathsf{S}) = [1], \qquad \text{fields}(\mathsf{AX}(c;x,y)) = [c+2,y,x]. \tag{4}$$

The literal source is compared against these exact selectors and inherited closed bodies. Swapping the two small bodies is not described as preserving every candidate number's meaning; it deliberately changes the enumeration order.

3.3 Soundness with a protected stack prefix

Call a tuple safe above a numerical tail v if t, h, and its explicitly protected stack heads are $\mathcal{K}_{\leq}$ theorems. The tail need not be safe. Transfer is sound whenever it has an available protected head: both of its halt tests inspect theorem-valued fields for code 1, and its nonhalting result moves two theorem codes. Failed MP creates truth; successful MP applies the literal inference rule and adds a truth head. AX introduces allowed rows, a dominated rho8 instance because $U \leq U + x$, a generalization of the old theorem cache, and the saved theorem top. Hence AX adds ten protected heads and preserves safety.

This invariant is independent of candidate well-formedness. Section 5 proves that the actual schedule never consumes the unknown tail of the initialized stack.

4 A complete derivation compiler

We give the inherited transfer compiler explicitly, because a safe-run invariant alone proves only the soundness half. The strong contract is: for each derived theorem code Z, there is a finite command block $\mathcal{B}_Z$ which, for every natural t, w, starts at $(t, w, 0, 0)$ and either halts or returns exactly $(Z, w, 0, 0)$. The arbitrary old stack is preserved.

Put $D = \mathsf{AX}(0; 0, 0)$ and define

$$\mathsf{H} = D; \mathsf{S}^{11}, \qquad \mathsf{PUSH} = D; \mathsf{S}^{9}; \mathsf{MP}(0).$$

After nine transfers following D, the tuple is $(L, [t; w], 0, Q_d(h))$. Since the cached quantifier has tag two, the last MP in PUSH fails by literal tag inequality. Therefore

$$\mathsf{PUSH}(t, w, d, h) = (0, [t; w], 0, 0)$$

for all raw inputs, including $t = 1$ or $h = 1$, with no halt. After ten transfers following D, the tuple is $(Q_d(h), w, 0, t)$. The eleventh tests t and pops precisely one old head. Consequently

$$\mathsf{H}(t, [x; v], d, h) = \begin{cases} \text{halt}, & t = 1, \\ (t, v, 0, x), & t \neq 1. \end{cases} \tag{5}$$

This is a top-preserving pop with a top-falsity guard, not the old NIP interface.

4.1 Truth and parameter setting

Define

$$\mathsf{RESET} = \mathsf{MP}(1); \mathsf{AX}(1; 0, 0); \mathsf{S}^{10}; \mathsf{MP}(0).$$

From $(t, w, 0, 0)$, the first MP fails because $I(t, 1) > 0$. AX then has $A = c = 1$, so its gate is zero and its saved top is zero. Ten transfers expose $Q_1(0)$ with cache zero. The last MP fails because $Q_1(0) > 0$. Thus RESET returns $(0, w, 0, 0)$ and provides $\mathcal{B}_0$.

When top, counter and cache are zero, let $J(0)$ be the empty word and $J(u) = \mathsf{MP}(u)$ for $u > 0$. It sets only the parameter to u. The empty zero case is essential: $\mathsf{MP}(0)$ would succeed by $I(0, 0) = 0$ and push an extra zero.

4.2 Axiom selection

The following blocks return the indicated theorem with the original stack restored and parameter/cache zero:

$$\begin{aligned} &R_j(a,b,c),\ 1 \le j \le 7: && \mathsf{RESET}; \mathsf{AX}(c;a,b); \mathsf{S}^{j+1}; \mathsf{H}^{9-j}, \\ &L(a,b,c,u),\ u \le a: && \mathsf{RESET}; J(u); \mathsf{AX}(c;a-u,b); \mathsf{S}^{9}; \mathsf{H}, \\ &G(a,b,c) \text{ enabled}: && \mathsf{RESET}; \mathsf{AX}(c;a,b); \mathsf{H}^{10}. \end{aligned} \tag{6}$$

For the first row, the transfers place the selected row on top, leaving exactly $9-j$ new entries above the old stack. Each H preserves that selected top and removes one entry. The last removed entry is saved truth, so the cache finishes at zero. The second row similarly selects the eighth head. The third keeps the gate itself on top while removing all ten generated heads. Each selected top is nonfalsity by its literal constructor tag.

4.3 Generalization

Given $\mathcal{B}_Z$, use

$$\mathcal{B}_{Q_i(Z)} = \mathcal{B}_Z; \mathsf{PUSH}; J(i); \mathsf{S}; \mathsf{AX}(i;0,0); \mathsf{S}^{10}. \tag{7}$$

After the first transfer, the state is $(0, w, i, Z)$. It tested old cache zero, rather than Z. AX has $U = A = c = i$, so its gate is zero and its ninth new head is exactly $Q_i(Z)$, followed by saved zero. Ten shifts return the required tuple. This calculation covers binder zero and even the formal premise $Z = 1$.

4.4 Modus ponens and its zero alias

For premises A and $I(A,B)$, compile the major premise first:

$$\begin{aligned} \mathcal{B}_B = \mathcal{B}_{I(A,B)}; \mathsf{PUSH}; \mathcal{B}_A; \mathsf{PUSH}; \mathsf{S}; \mathsf{S}; \mathsf{MP}(B); \\ \mathsf{AX}(B;0,0); \mathsf{S}^{11}. \end{aligned} \tag{8}$$

If neither premise block halts, the two saved premises give stack $[A, I(A,B); w]$. Two transfers return $(A, w, 0, I(A,B))$, unless $A = 1$ already causes a permitted halt. MP succeeds and returns $(B, [0; w], B, 0)$. AX has zero gate because $A = c = B$, and saves B above the intentionally added zero. Ten transfers put B in cache; the eleventh either halts on $B = 1$ or returns $(B, w, 0, 0)$. When $A = B = 0$, the major premise is raw zero, the MP success and zero push are intentional, and the final transfer removes exactly that zero. Numerical collapse of zero-pairing notation invalidates no equation.

Induction on a finite derivation proves the strong contract. Finite proof DAGs may be expanded to trees; compiling the major first introduces no dependency cycle. If 1 is derived, $\mathcal{B}_1; \mathsf{H}$ halts from every $(t, w, 0, 0)$.

4.5 Cofinal enumeration and completeness

Let $E(\text{fields}, r)$ be right-folded Cantor pairing. Encode a continuation by

$$\text{tail}([]) = 0, \qquad \text{tail}(k :: L) = 1 + E(\text{fields}(k), \text{tail}(L)).$$

A positive continuation is decremented before its first field is read; a newly fetched candidate is read without that outer successor. Every nonhalting continuation drains strictly, since every right projection is at most its argument and therefore the next residual is at most $p - 1 < p$.

Fix $R = \mathrm{tail}(\mathcal{B}_1; \mathsf{H})$. For every Y, candidate

$$N_Y = P(2, P(Y, P(1, R))) \geq Y \tag{9}$$

begins with $\mathsf{AX}(0; 1, Y)$ and then executes that refutation suffix. AX resets parameter and cache from any retained tuple. The strong block is universal in its resulting top and stack. Thus successful candidates are cofinal above every finite startup cutoff. Exact arithmetic-macro termination and the scheduler's one-unit candidate increment ensure that a chosen successful candidate is reached unless an earlier one already halts. There is no fairness assumption or bounded proof-search test hidden in this argument.

5 One protected head suffices for the new schedule

Assign conservative credits

$$c(\mathsf{S}) = -1, \qquad c(\mathsf{MP}) = 0, \qquad c(\mathsf{AX}) = 10.$$

The optional successful-MP push is deliberately ignored. Only the transfer consumes a protected head, and AX constructs all ten heads before another command starts. The needed initial credit is the negative of the smallest cumulative command-prefix balance.

Let f be the right Cantor projection and $T_s = s(s+1)/2$. Define $R(0)$ to be empty and $R(r) = W(r-1)$ for $r > 0$. If $P^{-1}(n) = (a, y)$, the new wire format gives

$$W(n) = \begin{cases} \mathsf{MP}; R(f(y)), & a = 0, \\ \mathsf{S}; R(y), & a = 1, \\ \mathsf{AX}; R(f(f(y))), & a \geq 2. \end{cases} \tag{10}$$

These words specify command types and field counts; actual payloads affect theorem computation but do not worsen conservative stack credit.

Let $b(n)$ be the total balance of $W(n)$ and $B(k) = \sum_{n<k} b(n)$. Let $g(0) = 0$ and $g(r) = b(r-1)$ for $r > 0$.

Lemma 5.1 (Prefix-sum pullback)**.** *If a sequence $u(0), u(1), \ldots$ has nonnegative sums on every finite initial segment, then so does $u(f(0)), u(f(1)), \ldots$.*

Proof. The sequence $f(n)$ is the concatenation (0), $(0,1)$, $(0,1,2)$, and so on. A prefix sum of the pulled-back sequence is a sum of complete original prefix sums plus one further partial prefix sum. Each is nonnegative. The statement may be iterated for $f \circ f$. □

Lemma 5.2 (Length and continuation debt)**.** *The word $W(n)$ has length at most $n+1$. Every prefix of $R(r)$ has balance at least $-r$.*

Proof. Every nonempty recursive continuation in (10) is $W(v-1)$ with $v \leq y \leq n$. Induction gives length at most $1 + v \leq n+1$. A continuation with at most r commands loses at most r credits because every command has credit at least -1. □

Theorem 5.3 (Exact one-head bound)**.** *For the schedule of all candidates in increasing order, the minimum conservative prefix balance is -1. Starting at candidate cutoff one with empty proof pointer therefore needs exactly one protected head in this conservative accounting. This is a sufficient bound for actual execution, not a claim of a necessary minimum after successful-MP pushes are counted. Every Cantor diagonal $s \geq 2$ is prefix-safe relative to its initial balance.*

Proof. The initial words are $W(0) = \mathsf{MP}$, $W(1) = \mathsf{S}$, and $W(2) = \mathsf{MP}$, giving boundary balances $0, 0, -1, -1$. The loss -1 is attained by candidate one, so the claimed initial credit is necessary for this conservative schedule.

Induct simultaneously over diagonals that $B(k) + k \geq 0$ for all already reached boundaries, that $B(T_s) \geq -1$, and that all previously completed diagonals $s \geq 2$ were prefix-safe. Since

$$\sum_{r<m} (g(r) + 1) = B(m-1) + m \geq 0 \quad (m \geq 1),$$

we use only prefix lengths $m \leq s - 1$ already covered by the induction hypothesis. The pullback lemma, applied twice to the needed lengths $k \leq s - 1$, gives

$$\sum_{y<k} g(f(f(y))) \geq -k.$$

On diagonal $s \geq 2$, the first $s - 1$ candidate selectors are AX, in order $y = 0, \ldots, s - 2$. Their total balance after k such candidates is at least $10k - k = 9k$. Within the AX candidate at position y, the previous completed AX candidates contribute at least $9y$, its leading AX adds ten, and its continuation loses at most $f(f(y)) \leq y$. Every prefix after the leading AX is therefore at least $8y + 10 > 0$ relative to the diagonal start. The empty prefix before this AX is bounded separately by $9y \geq 0$.

The penultimate candidate is transfer, with continuation $R(s - 1)$. The prefix before it has at least $9(s-1)$ credit; including its transfer and any continuation prefix gives at least $9(s-1) - 1 - (s-1) = 8s - 9 \geq 7$. The final candidate is MP, with continuation $R(f(s))$. The completed penultimate candidate has the same lower bound, and $f(s) \leq s$, so its prefixes are at least $7s - 9 \geq 5$. Thus the entire diagonal is prefix-safe and its final balance is nonnegative relative to its start.

All recursive candidate indices used in these continuation/pullback premises are smaller than the current diagonal's first index; for $s \geq 2$, $f(f(y)) \leq y \leq s - 2 < T_s$ and $s - 1 < T_s$. This justifies the induction rather than assuming the conclusion for the current candidates. The same bounds close $B(k) + k \geq 0$ at newly reached boundaries. Consequently no later prefix drops below the initial minimum -1. □

An independent standard-library computation corroborates the recurrence on 500,000 candidates, 1,000 explicitly expanded words, and 997 complete diagonals. Those finite counts are not the proof of the theorem; the preceding induction supplies the unbounded statement. The exact minimum differs from the earlier ordering $\mathsf{S} = 0, \mathsf{MP} = 1$, whose initial exceptional words require two protected heads from cutoff one.

6 Exact source and whole-tape implementation

The source language has nine natural-number registers and primitives increment, guarded decrement, structured break, structured continue, and halt. Each leaf carries its binary tree address. Registers have logical order

$$(N, T, W, A, D, H, X, Y, L),$$

where N is the next candidate, L the remaining proof list, and X, Y are scratch registers. A duplicated source subtree is represented by one shared decoder role, but that role may interact with arithmetic helper rows. Source subtree count alone is therefore insufficient to certify a table.

The portable builder reconstructs the exact new source from the pinned parent and checks both the ordered primitive effects and every guarded control target outside the changed closed bodies.

Two independent resolvers are used: arithmetic on binary prefixes, and subtree-interval/ancestor traversal. The source-level proof identifies the two relocated closed bodies with the inherited MP and composite-transfer contracts, the changed selector with (4), and every untouched AX/scheduler component with its inherited contract. Scratch restoration is part of each macro contract.

6.1 Frames

Let $B(v) = 0\,1^{v+1}$ and $C(v) = 1\,0\,1^{v}$. A frame begins at head h in native dispatcher state 84 with marker 1 at $h-1$, source address p starting at h, zeros through a header 1 at $h+64+g$, and then primary blocks in physical order

$$(W, L, T, D, A, Y, N, H, X),$$

followed by a nonempty finite list of arbitrary natural-valued spare blocks. Here $g \geq 0$, $|p| \leq 64$, and the entire exterior is zero. The addressed instruction is determined by p followed by zero padding. The primary machine appends spare value one on every increment.

Theorem 6.1 (Quantified primitive simulation)**.** *For every source occurrence and every applicable decrement guard, every natural primary-register tuple, every natural gap, and every nonempty finite natural spare list, the exact machine executes a finite positive number of transitions from its frame to the prescribed successor frame, or to the binary halt exactly for the source halt.*

Proof. The included symbolic verifier freshly derives all 6,209 cases for the literal table. It represents whole tapes by finite symbolic words, affine unary runs, finite lists of blocks, and zero exteriors. Its rules are literal transitions, induction on a unary run with the requisite scan self-loop, and induction on a finite block list after a one-block lemma has been proved from the exact rows. This is an executable ordinary mathematical proof system, rather than testing a finite set of register values.

Four block schemas preserve or convert B and C while scanning left or right. Their complete one-block traces are retained. Each crosses a block of value v in $v+2$ transitions. The relation $C(S)1 = 1B(S)$ supplies the list concatenation identity. Thirteen affine slots represent the constant, gap, nine independent register parameters, first spare value, and total width of the remaining spare list. A positive decrement uses one plus its selected parameter; a zero guard fixes that register to zero.

Every derived duration has nonnegative integer coefficients and a positive constant, with minimum constant 10. The proof uses 1,610,781 symbolic rule applications and covers all 430 transition rows together with the block schemas. The case census and exact table/source hashes bind these statements to the delivered files. Each case closes at the asserted whole-tape endpoint, including the infinite zero exterior. □

This full runtime proof is distinct from the finite decoder-prefix allocation check. A proposed allocation can pass all source prefixes yet break an arithmetic helper; such failures are retained in the predecessor research. The primary presented here passes the stronger check on its own exact source. The compact single-guard alternatives do not; a finite routing pass is not substituted for their missing proof.

7 Blank startup and the arbitrary tail

The literal blank execution reaches native dispatcher 84 at head -2 after exactly 68,442 transitions, with source ordinal 55 and address `110100100101010`. Its logical registers are

$$(N, T, W, A, D, H, X, Y, L) = (1, 11, 2, 0, 0, 0, 0, 0, 0),$$

its gap is zero, and it has 59 unit-valued spares. Independent native and dense interpreters reconstruct the entire one-cell support and agree exactly with the frame generated from these data. All other tape cells are zero.

This is an internal AX frame, not a safe source root: treating arbitrary intermediate top 11 or stack 2 as theorems would leave a logical gap. The bridge is an exact finite source computation. Starting at AX entry ordinal 25 with

$$(N, T, W, A, D, H, X, Y, L) = (1, 0, 4, 0, 0, 0, 0, 0, 0),$$

the inherited AX prefix reaches the displayed ordinal-55 store after 174 primitive steps. The full trace is included and replayed independently; the two source expressions agree literally on this prefix.

Complete that AX symbolically under its all-natural contract. Its initial top and cache are truth, and its numerical tail is arbitrary stack value 4. It constructs ten theorem heads above that tail and returns with zero top, parameter, cache, scratches, and proof pointer, with next candidate one. The actual seed is therefore safe above an unknown tail after AX completion; no claim that tail 4 is a good stack is required.

The one-head theorem proves that all later command prefixes consume at most one of these ten initial credits before fresh AX credits are available. At least nine protected theorem heads remain above the unknown tail in the conservative account. A transfer always has a protected head when it is executed. The theoremhood invariant therefore applies to every subsequent actual command and every halt. Conversely, the strong compiler and cofinal family (9) force a halt if a native refutation exists.

Positive-duration whole-tape simulation composes these source facts with the actual blank prefix. Each nonhalting source block finishes in finite positive time; no additional binary halt can occur inside a proved nonhalting block. Infinite source execution thus gives infinite binary execution, while a source halt reaches the binary halt. This closes the exact-table equivalence stated in Section 1.

8 A calculus reduction: one orientation of quantifier commutation

The numerical calculus permits a useful reduction independently of any particular binary allocation. Only one direction of quantifier commutation is needed for each pair of distinct labels. The result preserves the exact native theorem codes, so it avoids any translation of the code-dependent seed bundle.

8.1 Definitions and hypotheses

Write

$$P(u, v) = \frac{(u + v)(u + v + 1)}{2} + v, \qquad I(U, V) = 4P(U, V), \qquad Q_a(U) = 4P(a, U) + 2,$$

and let the native falsity code be $F = 1$. In the formulas below, $U \to V$ denotes the native code $I(U, V)$, and

$$\neg U = I(U, F), \qquad \Box_a U = Q_a(U), \qquad \Diamond_a U = \neg \Box_a \neg U.$$

These are numerical constructors. Parameters denote arbitrary natural numbers, and no assumption that a parameter decodes as an ordinary formula is needed. In particular, $I(0, 0) = 0$ is retained literally.

The proof uses the classical propositional interface K, S, and $\neg\neg U \to U$, together with numerical modus ponens, *theorem-only* generalization, and the retained rows

$$\rho_2(a; U, V) : \Box_a(U \to V) \to (\Box_a U \to \Box_a V), \tag{1}$$

$$\rho_6(a; U) : \Diamond_a \Box_a U \to U. \tag{2}$$

For the current dominated calculus, the classical interface is supplied by the inherited dominated-ρ_8 calibration together with the retained ρ_6 row. It is an explicit premise of the present reduction, rather than a new consequence of a finite truth-table calculation. The falsity interface can be seen explicitly: K gives $F \to \Diamond_i \Box_i X$, and composition with ρ_6 gives $F \to X$. The inherited classical implicational interface then yields double-negation elimination. No meaning is assigned to code 1 by a truth-table test.

Theorem. For each unordered pair of distinct natural labels, select one orientation (a, b). Retain the commutation axiom only in that orientation, for every natural matrix code X:

$$\Box_a \Box_b X \to \Box_b \Box_a X. \tag{3}$$

Keep all other axioms and rules unchanged. Then every omitted commutation instance is derivable at exactly its original native code. Consequently the restricted and unrestricted calculi derive precisely the same natural numbers. One possible selection is $a < b$; diagonal instances require no commutation axiom.

8.2 Finite derivation of the opposite direction

We first record two admissible transformations on *closed* theorems. Normality and theorem-only generalization take $U \to V$ to $\Box_i U \to \Box_i V$. Classical contraposition then gives the corresponding diamond monotonicity $\Diamond_i U \to \Diamond_i V$.

Applying (2) to $\neg U$, taking its contrapositive, and eliminating double negations gives the unit

$$U \to \Box_i \Diamond_i U. \tag{4}$$

More explicitly, contraposition initially yields $\neg\neg U \to \neg\neg(\Box_i \Diamond_i U)$. Compose this with double-negation introduction at U and elimination at $\Box_i \Diamond_i U$ to obtain (4). It follows that a closed theorem $\Diamond_i U \to V$ gives the closed theorem

$$U \to \Box_i V. \tag{5}$$

Indeed, generalize the given closed theorem, apply normality, and compose with (4). No open hypothesis is generalized.

Fix arbitrary numerical labels a, b and matrix code X, and set

$$Y = \Box_b \Box_a X.$$

Use the retained orientation (3) only at the matrix $\neg Y$. Classical contraposition, with double-negation transport inside the two boxes, yields

$$\Diamond_b \Diamond_a Y \to \Diamond_a \Diamond_b Y. \tag{6}$$

The double-negation transports in this step are important at the level of literal numerical codes: the nested diamonds are expanded as $\neg\Box_b\neg\neg\Box_a\neg Y$ and its counterpart, rather than identified with the unnormalized expressions by fiat.

Now compose three closed theorems:

$$\Diamond_b\Diamond_a Y \longrightarrow \Diamond_a\Diamond_b Y \longrightarrow \Diamond_a\Box_a X \longrightarrow X.$$

The first is (6). The second is diamond monotonicity applied to $\Diamond_b\Box_b(\Box_a X) \to \Box_a X$. The third is (2) at (a, X). Apply (5) first at label b, then at label a. This gives, successively,

$$\Diamond_a Y \to \Box_b X, \qquad Y \to \Box_a\Box_b X.$$

Substituting the definition of Y proves the missing orientation

$$\Box_b\Box_a X \to \Box_a\Box_b X.$$

For $a = b$, the commutation formula is an identity $Z \to Z$, already derivable from K and S. Every leaf of an unrestricted finite proof can therefore be replaced by a finite restricted proof. The converse inclusion is immediate because every retained axiom was available before. The replacement leaves every other native axiom code, including the gated seed bundle, unchanged.

8.3 Certificate and separate audit

The supplied generator `ordered_commutation.py` writes a 600-node finite schematic Hilbert certificate. Its two closed outputs are the reversed commutation theorem and the unit (4). The generator checks its output using the pinned inherited `hilbert.py` reader.

A separately implemented standard-library reader, `independent_ordered_audit.py`, imports neither that generator nor its proof kernel. It reconstructs the allowed axiom schemas and checks every parent index, modus-ponens match, open-hypothesis set, generalization premise, and final formula. Its actual result is `INDEPENDENT_ORDERED_COMMUTATION_AUDIT_PASS`. The audit establishes the following exact finite facts:

- All 600 stored nodes pass the reader. The dependency graph of the two outputs contains 488 nodes and no hypothesis nodes.
- There is exactly one oriented-commutation axiom instance, at node 70. Its ordered labels are (a, b) and its matrix is precisely $\neg(\Box_b\Box_a X)$. No opposite-direction instance is admitted as an axiom.
- All five generalization nodes have closed premises.
- The only axiom interfaces used are K, S, double-negation elimination, ρ_2, ρ_6, and the stated orientation of commutation.

The certificate uses syntactic tree equality as a sufficient condition for equality of instantiated numerical codes. It never infers constructor injectivity or treats schematic instantiation as a new object-level substitution rule. Evaluating each schematic parameter at arbitrary naturals turns the checked graph into a finite native proof; the zero alias causes no exception.

This is an executable finite proof certificate and a separate reader audit. It is not a new Lean or Rocq kernel endpoint. The derivations of the K/S/classical interface from dominated ρ_8 and retained ρ_6 remain the identified inherited premise; the present certificate does not recheck that larger certificate.

8.4 Scope for machine optimization

Because the restriction preserves every native theorem code, the existing consistency calibration transfers to a source that enumerates this restricted commutation family and all the other required rows. This conclusion supplies a logical interface for source optimization; it does not by itself supply a smaller binary machine.

In particular, imposing $a \leq b$ on the *entire* present AX command is not justified by this theorem. Its other schemas share the same parameters. The existing equality-reflexivity argument, for example, uses $\rho_1(f(x, x), x)$ with a fresh merged-equality binder $f(x, x) > x$, and such an instance lies outside that proposed global restriction. Recovering those other rows and preserving the exact seed-bundle calibration would require additional proofs. A concrete state saving also requires a source, an allocation, an arbitrary-value runtime proof, and an exact blank-start connection.

9 Research toward 214 and exact failure boundaries

The packet keeps unsuccessful reductions separate from the delivered machines. They are useful primarily because they show why a plausible local saving does not yet supply a smaller consistency machine.

9.1 Runtime folding and initialization

The incoming 214-state runtime fold identifies the relevant decoder roles of native states 88 and 301, with a changed helper row. Its valid runtime frames can be handled because a following marker bit has a prescribed value. The blank initialization has no such invariant. We tested its composition with zero-valued appended spares, and reconstructed exact outward-ray traces for every existing initial state. This does not make the table a consistency machine: a proved unconditional outward ray cannot encode Con(ZFC) in the required way.

A stronger finite obstruction checks all one-bit configurations reachable after one transition from a hypothetical extra initializer state, while leaving the old core rows fixed. For both appended-spare policies, the recorded configurations either halt or reach a literal outward self-loop. Thus every such one-state initializer has a finite certified outcome: an explicit halt or an unconditional outward ray. It supplies no calibrated proof-search startup for that unchanged core. Changing other core rows remains outside this obstruction.

9.2 A shorter transfer and context-sensitive primitive entries

The direct clear/copy/NIP body reduces the transfer from 78 to 35 primitive occurrences and retains 201 distinct source subtrees in a 4,732-occurrence source. Its initial 215-state allocation passes source-prefix routing checks but fails the stronger whole-tape theorem. A concrete test with top zero and cache one decrements the cache instead of the top at the new clear instruction. A marker-dependent physical entry intended for one decrement context selects a different register in this context.

A reassociation that preserves the complete source control graph shifts the first failure to a structured return instruction. Exhaustively trying 860 existing entry actions at that one location did not close the bounded symbolic proof. This is evidence about that finite repair family, not a theorem that the short command cannot be compiled in 215 states. A valid 69,744-step partial bootstrap for one zero-spare composition is also insufficient: the runtime failure remains. No short-transfer allocation is used in the final equivalence theorem.

A new physically verified, logically open variant. A further repair inserts a shared increment of scratch register X before the problematic return. It has 201 distinct subtrees, 4,733 primitive occurrences, and 215 working states. With zero-valued appended spares and native start 74, it passes all 6,152 arbitrary-natural runtime cases, using 1,617,669 symbolic rules and covering all 430 rows; its

69,744-step startup matches a synthetic AX prefix. This is a positive physical result for the short transfer.

The extra increment is not semantically dead: after either MP or AX it leaves $X = 1$, while transfer tag zero leaves $X = 0$. The following Cantor read acts on $n + 1$, and a candidate refill can advance the counter by two. The existing credit and cofinality proofs therefore do not apply unchanged. The exact source, table, full runtime receipts, and contract analysis are included under `research/short-transfer`; no equivalence with Con(ZFC) is asserted for this second machine. The outstanding problem is a complete logical analysis of its state-dependent enumeration, or a physical repair restoring a clean scratch register without adding a state.

9.3 Shorter delimiter scanner

A scanner reduction would be available if the source addresses could be made to avoid six consecutive zeros while keeping the decoder cost low enough. A targeted closed-subtree reassociation reduced the number of offending occurrences from 37 to 2 with 204 distinct subtrees. Two further rotations produced a genuine maximum-zero-run-five source with 206 subtrees and an independently identical guarded source graph. This is a valid source transformation, but its five extra source nodes do not justify counting the proposed one-state scanner saving as a net physical improvement. No smaller table is claimed from it.

9.4 Finite local-fold searches

A focused all-tape diagram search on the delivered table examined 46,010 ordered state pairs and 629,160 rewrite specifications, with radius five and fuel 97, and found no successful fold in that family. Related searches were run on experimental short-transfer allocations. The retained records state the exact input hashes, ordered state pairs, candidate rewrite family, finite window radius, and proof fuel. A negative search result excludes only the recorded finite proof-search family; it is not a Busy Beaver lower bound or a proof that 214 states is impossible. The concrete failed startup compositions have stronger scope because their literal outward rays are proved, rather than merely not found within a simulation bound.

10 Reproduction and proof status

The ZIP is self-contained for reconstruction, the stated finite certificates, source correspondences, exact blank execution, and the full primitive-runtime proof. Python 3.10 or later and the standard library suffice for its supported verification entry points. Use assertions; do not run with `-O`.

From the extracted packet root:

```
python3 -B verify.py
python3 -B verify.py --runtime --out /new/output/directory
```

The default checks the manifest, reconstructs all literal artifacts, validates dense/native renaming and distinction witnesses, replays complete startup and source-prefix computations, checks the stored runtime census, and reruns the logical/scheduler certificates. The second command additionally re-derives every arbitrary-natural primitive case from the exact table; the stored census is not substituted for fresh derivations.

For the report:

```
cd report
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The new assertions are ordinary mathematical proofs and executable proof checks. No new exact-table Lean/Rocq theorem or internal-PA endpoint has been compiled here. The final ZFC equivalence uses the explicit inherited calibration of $\mathcal{K}_{\leq}$; it does not rely on a finite absence of halting, a theorem-prover timeout, or an assumption that ZFC proves its own consistency. The honest numerical outcome is a completed, distinct 215-state candidate and several precisely scoped routes that have not yet produced 214.

A Complete dense transition table

The table gives both read-symbol actions for every working state. An entry wDq means write w, move in direction D, and enter q. Symbol H is halt 215. All entries belong to the same machine whose exact bytes and canonicalization are included in the archive.

q	read 0	read 1	q	read 0	read 1	q	read 0	read 1
0	1R91	1R0	72	0R79	1R80	144	0R83	1R87
1	0L1	1L2	73	1R0	1R63	145	0R156	1R12
2	0L3	1L2	74	0R74	1R81	146	0R82	1R141
3	0L5	1L2	75	0L143	1L139	147	0R161	1R165
4	1R6	0L4	76	0R84	1R85	148	0L66	1L139
5	0L9	1L2	77	0R88	1L131	149	0R146	1R199
6	0R6	0R10	78	0L107	1R56	150	0R156	1R113
7	0R83	1R187	79	0R48	0R33	151	0R96	1R96
8	0R14	1R13	80	0R90	0L41	152	0R48	0R32
9	0L15	1L2	81	0R92	1R174	153	0R68	1R200
10	0L16	0R10	82	0R95	1R12	154	0L207	1R95
11	0L26	1R128	83	0R87	1R96	155	0R165	1R176
12	0R19	1R20	84	0R203	1R21	156	0R19	1R167
13	0L88	1L31	85	0R115	1R99	157	0R170	1R162
14	0L11	0R22	86	0R148	1R112	158	0R156	1R95
15	0L23	1L2	87	0R101	1R102	159	0R124	1R161
16	1L1	0L1	88	0R79	1R78	160	0R96	1R159
17	0R24	0R32	89	0R100	1R171	161	0L7	1R140
18	0R198	1R197	90	0R104	1R105	162	0L108	1R170
19	0R28	1R29	91	0R65	0R0	163	0R144	1R169
20	0R30	1R31	92	0R27	1R106	164	0R193	1R28
21	0R24	0R59	93	0R179	0L204	165	0R158	1R141
22	0R33	1R22	94	1L109	1L94	166	0R175	1R175
23	0L34	1L2	95	0R19	1R110	167	0R178	1R121
24	0L35	0R0	96	0R60	1R60	168	0R143	1R162
25	0R36	1R25	97	0R164	1R154	169	0R75	1R60
26	0R127	1R37	98	0R66	1R186	170	0R143	1R87
27	0R38	0L39	99	0R104	0L204	171	0R103	1R95
28	0R40	1R41	100	0R116	1R117	172	0R138	1R83
29	0R42	1R43	101	0R71	1R58	173	0R172	1R75
30	0R44	1R45	102	0R57	1R118	174	0R107	1R214
31	0R21	1R46	103	0R21	1R119	175	0R140	1L147
32	0R47	1R32	104	0R21	0L10	176	0R96	1R51
33	0R32	1R33	105	0R61	1R41	177	0R165	1R51
34	0L49	1L2	106	0R120	1R8	178	0R17	1L130
35	1R35	1L1	107	0R100	1R121	179	0R76	1R115

q	read 0	read 1	q	read 0	read 1	q	read 0	read 1
36	0R50	1R36	108	0R148	1R183	180	0R105	1R123
37	0R13	1R52	109	0L204	0L94	181	1L39	1L109
38	0L11	0R36	110	0R123	1R103	182	0R168	1R156
39	0L4	1L53	111	0R206	1R180	183	0R148	1R114
40	0R24	0R25	112	0R162	1R178	184	0R11	1R212
41	0R13	0L16	113	0R146	1R144	185	0R189	1R98
42	0R55	1R56	114	0R157	1R197	186	0R144	1R195
43	0R57	1R58	115	0L130	0L1	187	0R154	1R188
44	0R24	0R54	116	0R38	1R72	188	0R172	1R182
45	0R60	1R61	117	0R61	1R131	189	0R64	1R7
46	0R62	0L16	118	0R61	1R132	190	0R148	1R126
47	0R25	1R47	119	0R70	0L16	191	0R153	1R111
48	0L35	0R63	120	0R14	1R131	192	0R150	1R122
49	0L74	1L2	121	0R21	1R133	193	0R89	1R87
50	0R16	1R50	122	0R145	1R175	194	0R184	0L204
51	0R145	1R141	123	0R135	1L20	195	0R202	1R108
52	0R67	1R68	124	0R129	1R177	196	0R148	1R166
53	0L39	1L53	125	0R201	1R168	197	0R196	1R157
54	0R22	1R54	126	0R75	1R173	198	0R138	1R136
55	0L11	0R50	127	0R17	1R194	199	0R192	1R18
56	0R60	0L16	128	0R75	1R134	200	0R87	1R205
57	0R69	1R70	129	0R12	1R139	201	0R160	1R190
58	0R61	1R71	130	0R30	1R69	202	0R125	1R149
59	0R54	1R59	131	0R38	1L123	203	0R44	1R99
60	0R48	0R25	132	0R151	0L16	204	0L4	0L204
61	0L28	1L72	133	0R152	0L16	205	0R193	1R138
62	0L116	1L30	134	0R158	1R137	206	0R161	1R207
63	0R73	1R63	135	0R24	0R47	207	0R208	1R179
64	0R96	1R148	136	0R138	1R177	208	0R141	1R209
65	1R181	1L53	137	0R191	1R93	209	0R142	1R210
66	0R142	1R143	138	0R95	1R156	210	0R211	1R86
67	0R76	1RH	139	0R172	1R163	211	0R185	1R162
68	0L193	1R31	140	0R12	1R156	212	0R213	1R37
69	0R40	1R62	141	0R87	1L163	213	0R12	1R37
70	0L88	1R77	142	0R87	1R155	214	0R127	1R97
71	0R55	1L78	143	0L150	1L160			

References

[1] V. Reshetnikov et al., *ProveIt: Smaller machines for Con(ZFC)*, pinned commit `ead95b5b336818a53c89b8c3d5249822e0570939`. Pinned research directory.

[2] ProveIt, *Candidate K215: ordinary proof and exact table*, same snapshot. Pinned K215 proof.

[3] ProveIt, *Dominated-rho8 consistency calibration*, Source 149 and the K218 dominated alternative. Pinned dominated-rho8 proof.

[4] *Transfer Pop 215 Materially Different*, incoming research packet and its preserved original report, at the same pinned snapshot. Pinned transfer packet.

[5] *Z215: Zero-Valued Spare Blocks and a Two-Truth Startup*, incoming research packet, same snapshot. Pinned zero-spare packet.

[6] E. Glazer, *k230-rocq*, original K230 numerical-calculus and machine formalization. Original K230 formalization.